Role: AI Match Mapping Engineer (Retrieval + Ranking, Sports Data)

Mission

Build and own an **AI-assisted sports match mapping engine** that links **OddsPortal (OP)** matches to **Bet365 (B365)** matches with high reliability.

We already have:

- Platform/Admin UI completed (Top-5 candidate review + human decisions).
- ~10,000 human-labeled decisions (Match / Swapped / No Match).
- APIs to fetch OP/B365 match feeds.

Your job is to finish the **AI-side pipeline** so it improves automation over time (Active Learning), not just output a “best guess.”

What’s Missing (Current Gap / Scope)

1) Inference Engine (Production-Ready)

- Always generate **Top-5 candidates** per OP match (even if scores are low).
- **Pre-filter** candidate pool: same sport + kickoff window (default ±30 minutes; configurable).
- SBERT retrieval (Top-10) → Cross-Encoder rerank (Top-5).
- Output standardized JSON:
    - `mapping_suggestion_id` (unique session id)
    - candidates_top5 with score, time_diff_min, swapped flag.

2) Safe Auto-Match Gate (Automation Rules)

Implement strict auto-approval logic:

- Auto-match ONLY if all pass:
    - **MinScore gate** (e.g., Score1 ≥ 0.90, configurable)
    - **Margin gate** (Score1 − Score2 ≥ 0.10, configurable)
    - **Category gate** (Women/U23/Reserves/B-team, etc.)
    - kickoff within tight window
- Otherwise return **NEED_REVIEW** (human queue).
- Must support **NO MATCH / abstain** behavior (don't force mapping).

3) Feedback Ingestion (Human Labels → Training Data)

- Build a feedback endpoint (or batch ingestion) to store platform decisions:
    - MATCH / SWAPPED / NO_MATCH (+ reason_code)
- Convert UI decisions into training records:
    - MATCH/SWAPPED → positive pairs
    - unselected Top-5 candidates → **hard negatives**
    - NO_MATCH sessions → store as unmatched + candidates as hard negatives
- Enforce **1:1 mapping constraint** to prevent duplicate confirmations.

4) Model Tuning (Not LLM Fine-tune)

We are NOT training an LLM.
We are tuning **retrieval/ranking models**:

- SBERT training using **MultipleNegativesRankingLoss** on (OP_text, B365_text) pairs.
- Cross-Encoder training as reranker using positives + hard negatives.
- Prevent label contamination:
    - never include the true positive in negative pool.

- Add rule-based **category tokens** to input strings: `[WOMEN]` `[U23]` `[RESERVES]` `[B-TEAM]`.

5) Offline Evaluation & Reporting (Mandatory)

Provide weekly and on-demand evaluation:

- Recall@5 / Recall@10 (does correct match appear in Top-5?)
- Precision@1 for AUTO_MATCH subset (is auto-confirm safe?)
- No-Match performance (false auto-matches when match doesn't exist)
- Drift monitoring (performance by sport/league/category).

Required Skills

- Strong Python + production ML engineering
- IR / Ranking systems: embeddings retrieval + reranking, candidate filtering
- SentenceTransformers (SBERT, CrossEncoder), PyTorch
- Data quality / labeling pipelines, hard negative mining
- Building evaluation pipelines (Recall@K, Precision, calibration/margin gates)
- Practical systems thinking: idempotency, state machines, DB constraints, logging

Nice to have

- Sports data normalization (team aliases, category parsing: Women/U23/Reserves)
- Experience with noisy real-world entity resolution / record linkage

---

Deliverables (First 7–10 Days)

1. **Inference API** returning Top-5 + mapping_suggestion_id + gates decision.

2. **Feedback ingestion** that consumes platform decisions and stores training-ready rows.
3. **Dataset builder** that turns 10k human labels into positives + hard negatives.
4. **Baseline offline report** (before/after tuning): Recall@5 and AUTO_MATCH precision.
5. **First tuned model** deployed behind a feature flag + rollback plan.

Success Metrics

- **Recall@5 ≥ 90%** for “match-exists” cases in targeted sports.
- **AUTO_MATCH precision ≥ 98%** (safe automation).
- Increasing automation rate over time (more AUTO_MATCH, fewer NEED_REVIEW) without data corruption.